

CST205- OBJECT ORIENTED PROGRAMMING USING JAVA

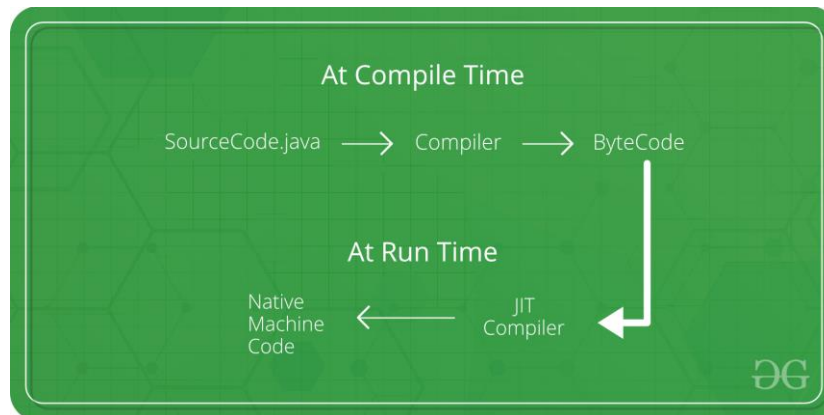
PART A

1 What is Just-In-Time compiler?

Java is compiled language. Javac is the most popular Java compiler. Java has a virtual machine called JVM which then converts byte code to target code of machine on which it is run. JVM performs like an interpreter. It doesn't do it alone. It has its own compiler to convert the byte code to machine code. This compiler is called Just In Time or JIT compiler.

The Just-In-Time (JIT) compiler is responsible for performance optimization of Java based applications run time. When a JIT(Just In Time) compiler is part of the JVM, selected portions of bytecode are compiled into executable code in real time, on a piece-by-piece, demand basis. JIT compiler compiles code as it is needed, during execution

- Not all sequences of bytecode are compiled -only codes that will benefit from compilation.
- The remaining code is simply interpreted.



2 Why Java is said to be a secure programming language?

- Every time a user compiles the Java program, the Java compiler creates a class file with Bytecode, which is tested by the JVM at the time of program execution for viruses and other malicious files.
- No explicit pointer so memory blocks cannot be accessed.
- Exception enables Java to capture a series of errors that helps developers to get rid of risk of crashing the system.
- Java's access-control functionality on variables and methods within the objects provide secure program by preventing access to the critical objects from the untrusted code.

3 Does Java support multiple inheritance? Justify your answer.

Java does not support multiple inheritance. When one class extends more than one classes then this is called multiple inheritance. For example: Class C extends class A and B then this type of inheritance is known as multiple inheritance. Multiple inheritance in java is implemented using interface.

Step 1: Define the interfaces

```
interface Interface1 {
```

```
void method1();  
}
```

```
interface Interface2 {  
    void method2();  
}
```

Step 2: Implement the interfaces in the class

```
public class MyClass implements Interface1, Interface2 {  
    public void method1() {  
        // implementation of method1  
    }  
    public void method2() {  
        // implementation of method2  
    }  
}
```

Step 3: Create an object of the class and call the interface methods

```
MyClass obj = new MyClass();  
obj.method1();  
obj.method2();
```

4 Why is the 'main' method in Java qualified as public, static, and void?

The public keyword is an access specifier, which allows the programmer to control the visibility of class members. When a class member is preceded by public, then that member may be accessed by code outside the class in which it is declared. In this case, main() must be declared as public, since it must be called by code outside of its class when the program is started. The keyword static allows main() to be called without having to instantiate a particular instance of the class. This is necessary since main() is called by the Java Virtual Machine before any objects are made. The keyword void tells the compiler that main() does not return a value.

5 Explain any three Byte Stream classes in Java.

Byte streams provide a convenient means for handling input and output of bytes. Byte streams are used, for example, when reading or writing binary data.

BufferedInputStream - Buffered input stream

BufferedOutputStream --Buffered output stream

ByteArrayInputStream Input stream that reads from a byte array

ByteArrayOutputStream -- Output stream that writes to a byte array

DataInputStream -----An input stream that contains methods for reading the Java standard data types

DataOutputStream -An output stream that contains methods for writing the Java standard data types

FileInputStream ---Input stream that reads from a file

FileOutputStream -----Output stream that writes to a file

6 What are Checked Exceptions? Give an example.

The classes which directly inherit Throwable class except RuntimeException and Error are known as checked exceptions. Checked exceptions are checked at compile-time. Checked exceptions are the exceptions defined by Java.lang that must be included in a method's throws list if that method can generate one of these exceptions and does not handle it itself.

Eg:

ClassNotFoundException-- Class not found.

NoSuchMethodException---- A requested method does not exist.

7 List any six Event Listener interfaces.

Listener is an object that is notified when an event occurs. It has two major requirements. First, it must have been registered with one or more sources to receive notifications about specific types of events. Second, it must implement methods to receive and process these notifications.

ActionListener--- Defines one method to receive action events.

AdjustmentListener---Defines one method to receive adjustment events.

ComponentListener-- Defines four methods to recognize when a component is hidden, moved, resized, or shown.

ContainerListener-- Defines two methods to recognize when a component is added to or removed from a container.

FocusListener-- Defines two methods to recognize when a component gains or loses keyboard focus.

ItemListener --Defines one method to recognize when the state of an item changes.

KeyListener --Defines three methods to recognize when a key is pressed, released, or typed.

MouseListener --Defines five methods to recognize when the mouse is clicked, enters a component, exits a component, is pressed, or is released.

MouseMotionListener---Defines two methods to recognize when the mouse is dragged or moved.

8 Explain any three special string operations in Java

Special string operations in Java operations include

1.the automatic creation of new String instances(object) from string literals

Java automatically constructs a String object for each string literal in our program,. So we can use a string literal to initialize a String object. We can use a string literal at any place where we use a String object.

Eg:

```
String st="abc";
```

is same as

```
char ch={'a','b','c'};
```

```
String st=new String(ch);
```

2.concatenation of multiple String objects by use of the + operator

- String concatenation is used to join two strings. The + operator can be used between strings to combine them. This is called concatenation.Operator + can be chained to concatenate many strings.

```
String age = "9";
```

```
String s = "He is " + age + " years old.";
```

```
System.out.println(s);
```

This fragment displays the string He is 9 years old.

3.the conversion of other data types to a string representation.

- We can concatenate strings with other types of data.If one of the operand of the + is an instance of String then compiler will convert other operand to its string equivalent.

```
String s = "four: " + 2 + 2;
```

```
System.out.println(s);
```

– This fragment displays

four: 22

- Operator precedence causes the concatenation of “four” with 2. So 2 is converted into string and “four: ” concatenates with string equivalent of 2.
- Then this result is then concatenated with the string equivalent of 2.

- Parentheses can be used for grouping integers and + to perform addition.

```
String s = "four: " + (2 + 2);
```

- Here parentheses is first computed. So (2+2) is 4 then string “four: ” is concatenated with that. So s contains the string “four: 4”

9 Summarize any three features of Swing API.

- Platform Independent

It is platform independent, the swing component that are used to build the programs are not platform specific.The behavior and appearance of Swing components is consistent across platforms, whereas AWT

components will differ from platform to platform. Thus Swing is platform independent whereas AWT is platform dependent.

- Swing Components Are Lightweight

This means that they are written entirely in Java and do not map directly to platform-specific peers.

- Swing Supports a Pluggable Look and Feel

SWING based GUI Application look and feel can be changed at run-time, based on available values.

- Swing uses MVC Architecture

Java's Swing components have been implemented using the model-view controller (MVC) model.

- Any Swing component can be viewed in terms of three independent aspects: what state it is in (its model), how it looks (its view), and what it does (its controller). Suppose the user clicks on a button. This action is detected by the controller. The controller tells the model to change into the pressed state. The model in turn generates an event that is passed to the view. The event tells the view that the button needs to be redrawn to reflect its change in state.

- Rich Controls

Swing is the latest GUI toolkit, and provides a richer set of interface components than the AWT.

- Customizable

Swing components can be given their own "look and feel". Eg: A button may have both an image and a text string associated with it. Also, the image can be changed as the state of the button changes.

- Manageable

It is easy to manage and configure. Its mechanism and composition pattern allows changing the settings at run time as well. The uniform changes can be provided to the user interface without doing any changes to application code.

10 Differentiate between Components and Containers in Swing.

Component

At the top of the AWT hierarchy is the Component class. Component is an abstract class that encapsulates all of the attributes of a visual component. All user interface elements that are displayed on the screen and that interact with the user are subclasses of Component. It defines over a hundred public methods that are responsible for managing events, such as mouse and keyboard input, positioning and sizing the window, and repainting. A Component object is responsible for remembering the current foreground and background colors and the currently selected text font.

Container

The Container class is a subclass of Component. It has additional methods that allow other Component objects to be nested within it. Other Container objects can be stored inside of a Container (since they are themselves instances of Component). This makes for a multileveled containment system. A container is responsible for laying out (that is, positioning) any components that it contains.

PART B

11 a) Explain Lexical issues in Java with examples.

Java programs is a collection of White spaces , Identifiers , comments , Literals , Operators , Separators and Keywords.

- White Spaces

Java is a free form language. This means that you do not need to follow any special indentation rules. In java , white spaces is a space , tab or new line.

- Identifiers

Identifiers are used for class names , method names and variable names. An identifier may be any descriptive sequence of uppercase and lowercase letters , numbers or the underscore and dollar sign.

Some valid identifiers are avg,\$test.

Some invalid identifiers are 2avg,no-1

- Literals

A constant value in java is created by using a literal representation of it. A literal can be used anywhere a value of its type is allowed.eg: 100,'c',"abc"

100 is an integer literal, 'c' is a character literal and "abc" is a string literal

- Comments

There are 3 types of comments in java. First is single line comment and the second one is multi line comment. The third type of comment is called documentation comment. It is used to produce an HTML file that documents your program. It begins with a /** and ends with a */.

- Separators

There are few symbols in java that are used as separators. The most commonly used separator in java is the semicolon ';' . some other separators are Parentheses '()' , Braces ' {} ' , Bracket ' [] ' , Comma ' , ' , Period ' . ' .

- Java Keywords

These keywords cannot be used as names for a variable , class or method. There are 50 keywords currently defined in Java.eg: true, false,class

b) Differentiate between function oriented and object oriented software design approaches using a suitable example.

1. Function Oriented Design

- In function-oriented design, the system is comprised of many smaller sub-systems known as functions.
- These functions can perform significant task in the system.
- The system is considered as top view of all functions.

- Function oriented design inherits some properties of structured design where divide and conquer methodology is used.
- This design mechanism divides the whole system into smaller functions, which provides means of abstraction by concealing the information and their operation.
- These functional modules can share information among themselves by means of information passing and using information available globally.
- Another characteristic of functions is that when a program calls a function, the function changes the state of the program, which sometimes is not acceptable by other modules.
- Function oriented design works well where the system state does not matter and program/functions work on input rather than on a state.

Eg: Banking process

Here withdraw, Deposit, Transfer are functions and that can be divided in to subfunctions again.

So, in FOD, the entire problem is divided into number of functions and those functions are broken down in to smaller functions and these smaller functions are converted in to software modules.

2. Object Oriented Design

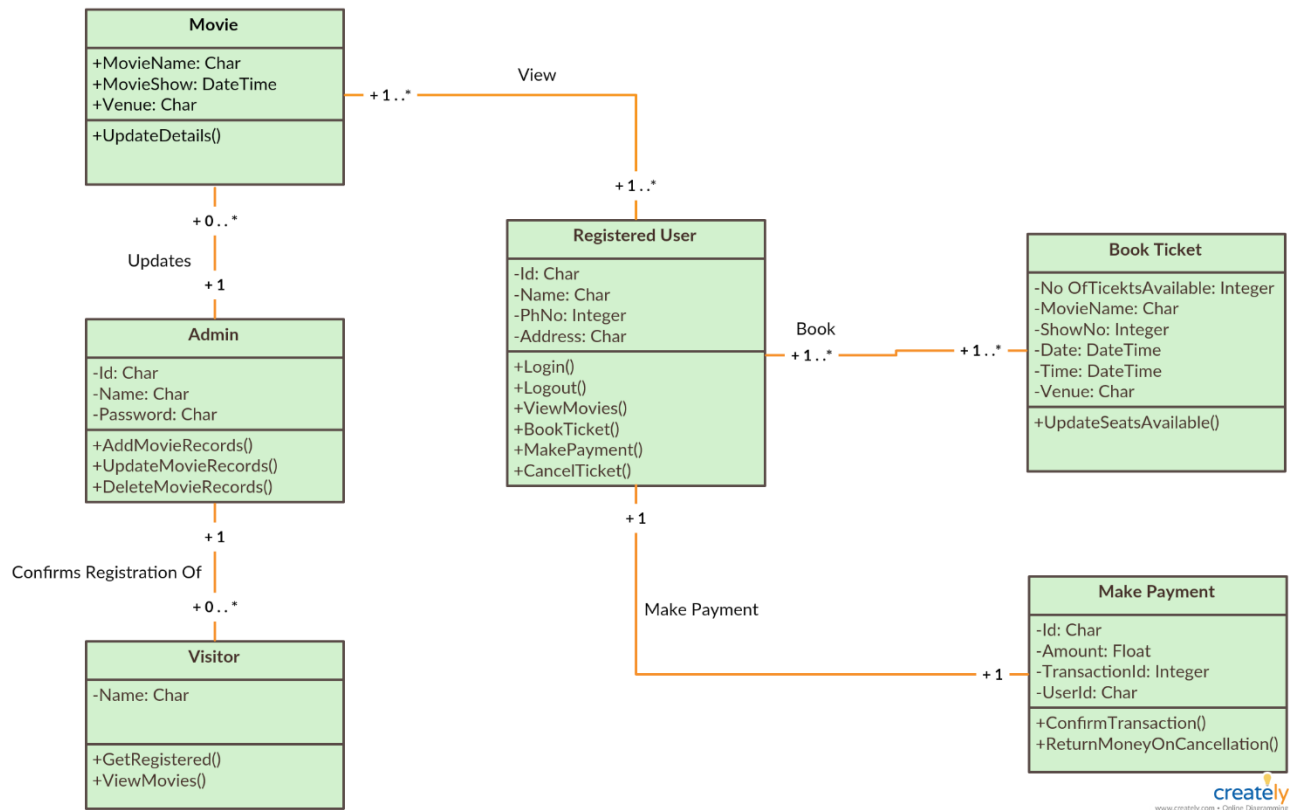
- Object oriented design works around the entities and their characteristics instead of functions involved in the software system.
- This design strategies focuses on entities and its characteristics.
- The whole concept of software solution revolves around the engaged entities.
 - OOD is based on Objects and interaction between the objects
 - Interaction between objects is called message communication.
 - It involves the designing of Objects, Classes and the relationship between the classes

Eg: Banking process.

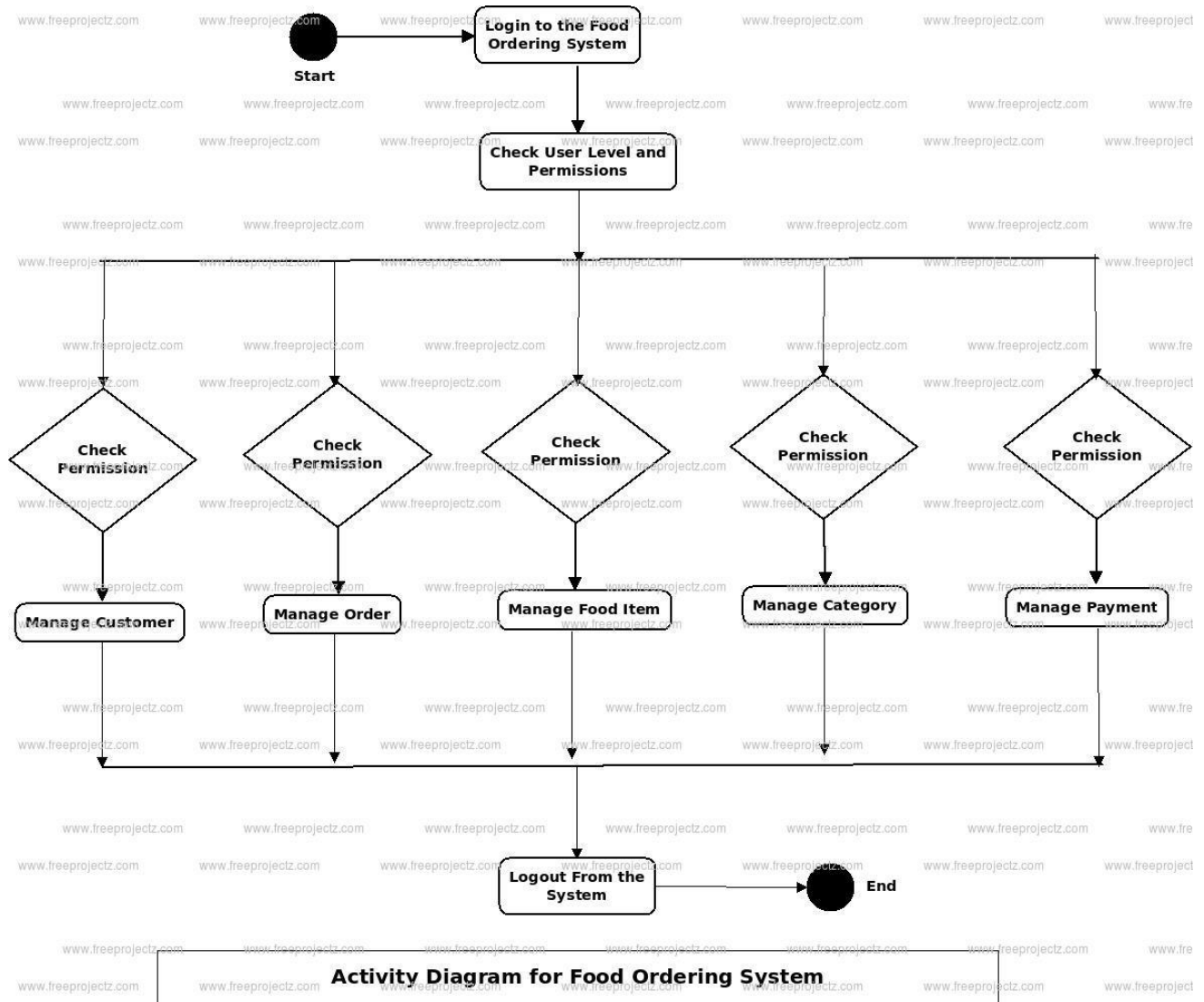
Here, customer, money and account are objects

Object-Oriented Approach	Function-Oriented Approach
The basic abstractions are not the real world functions but are the data abstraction where the real world entities are represented.	The basic abstractions, which are given to the user are real world functions.
Functions are grouped together on the basis of the data they operate since the classes are associated with their methods	Functions are grouped together by which a higher level function is obtained.
In this approach a state information is not represented in a centralised memory but is implemented or distributed among the objects of the system.	In this approach the state information is often represented in a centralised state memory.
We decompose in the class level	We decompose in the function/procedure level
Bottom up approach	Top down approach

12 a) Construct a UML Class diagram for Online Movie Ticket Booking System. The various entities involved in the system are Admin, Registered User, Visitor /Guest User, Movie, Book Ticket, Make Payment.

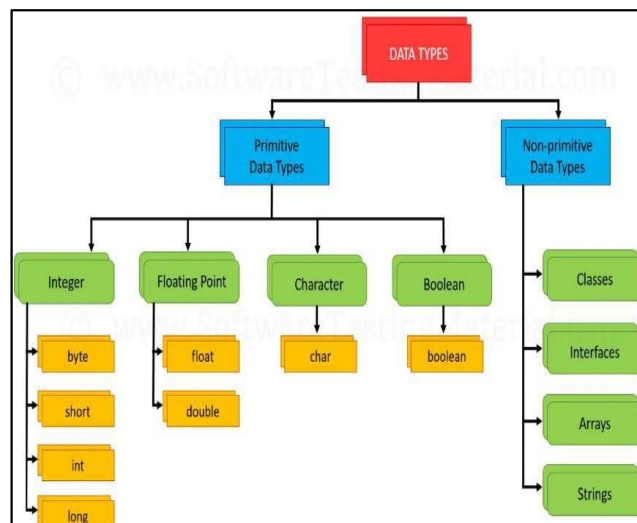


b) Construct a UML Activity diagram for Food Ordering System, which shows the flows between the activity of Order, Delivery, Food Item, Category, Payment.



13 a) Explain different data types in Java. Give examples.

- There are two types of data types in Java:
 - Primitive data types
 - Non-primitive data types



Primitive Data types are classified as:

- Integer type
 - Java supports 4 types of integers
 - byte - 1 byte
 - short -2 bytes
 - int -4 bytes
 - long-8 bytes
 - eg: int a=5;
- Floating point type
 - Java supports 2 kinds of floating point storage
 - float -4 bytes and double-8 bytes
 - eg float a=2.5f;
- Character type
 - Java uses Unicode to represent characters.
 - There are no negative chars.
 - char -2 bytes
 - Eg char c='x';
- Boolean type
 - **boolean** type is used for logical values.
 - It can have only one of two possible values, **true** or **false**.
 - This is the type returned by all relational operators
 - Eg boolean b=false;

b) Write a Java program to reverse bits of a given integer.

```
public class BitsDemo
{
public static void main(String[] args)
```

```

{
int input = 10;
System.out.println("Reversed actual bits of the given number");
int rev_input = 0;
while (input>0)
{
rev_input <<= 1;
if((int) (input & 1)==1)
{
rev_input^=1;
}
input>>=1;
}
System.out.println(rev_input);
}
}

```

Output

Reversed actual bits of the given number

5

14 a) Demonstrate how objects are passed as function parameters with a suitable example.

```

class Test {
int a, b;
Test(int i, int j){
a=i;
b=j;
}
boolean equals(Test o){
if(o.a== a && o.b== b)
return true;

```

```

else return false;

}}

class PassOb {

public static void main(String args[]){

Test ob1 = new Test(100, 22);

Test ob2 = new Test(100, 22);

Test ob3 = new Test(-1,-1);

System.out.println(ob1.equals(ob2));

System.out.println(ob1.equals(ob3));

}}

```

output

true

false

The equalsTo() method inside Test compares two objects for equality and returns the result. That is, it compares the invoking object with the one that it is passed. If they contain the same values, then the method returns true. Otherwise, it returns false. The parameter o in equalsTo() specifies Test as its type. Although Test is a class type created by the program, it is used in just the same way as Java's built-in types.

b) Write a Java program to find the frequency (count the occurrence) of each element in an integer array.

```

class Freq

{

public static void main(String[] args)

{

int arr[]={8, 3, 2, 2, 2, 5, 1};

//Array fr will store frequencies of element

int fr[] = new int [arr.length];

int visited=-1;

for(int i = 0; i < arr.length; i++)

{

int count = 1;

```

```

for(int j = i+1;j < arr.length;j++)
{
if(arr[i]== arr[j])
{
count++;
//To avoid counting same element again
fr[j] = visited;
}
}
if(fr[i]!=visited)
fr[i] = count;
}
//Displays the frequency of each element present in array
System.out.println(" Element - Frequency");

for(int i = 0; i < fr.length; i++)
{
if(fr[i]!= visited)
System.out.println(" "+arr[i]+ "- "+fr[i]);
}
}
}

```

Output

Element - Frequency

8- 1

3- 1

2- 3

5- 1

1- 1

15 a) Develop a Java package named 'evenpackage', with a class Even containing a static method that check whether a number is even or not, and returns that information. Import this package in another class and use to check a number is even or not.

```
package evenpackage;

public class Even{

public static void oddoreven(int num)

{

if(num%2==0)

System.out.println("Even");

else

System.out.println("Odd");

}

}

import evenpackage;

class B{

public static void main(String args[]){

Even obj = new Even();

obj.oddoreven(15);

}

}
```

b) Differentiate between try-catch, throw and throws keywords.

try-catch

To guard against and handle a run-time error, simply enclose the code that you want to monitor inside a try block. Immediately following the try block, include a catch clause that specifies the exception type that you wish to catch. A try and its catch statement form a unit. The scope of the catch clause is restricted to those statements specified by the immediately preceding try statement. A catch statement cannot catch an exception thrown by another try statement (except in the case of nested try statements). The statements that are protected by try must be surrounded by curly braces. (That is, they must be within a block.) You cannot use try on a single statement. The goal of most well-constructed catch clauses should be to resolve the exceptional condition and then continue on as

if the error had never happened.

throw

It is possible for your program to throw an exception explicitly, using the throw statement. The general form of throw is shown here:

```
throw ThrowableInstance;
```

Here, ThrowableInstance must be an object of type Throwable or a subclass of Throwable.

Primitive types, such as int or char, as well as non-Throwable classes, such as String and Object, cannot be used as exceptions. There are two ways you can obtain a Throwable object: using a parameter in a catch clause, or creating one with the new operator. The flow of execution stops immediately after the throw statement; any subsequent statements are not executed. The nearest enclosing try block is inspected to see if it has a catch statement that matches the type of exception. If it does find a match, control is transferred to that statement. If not, then the next enclosing try statement is inspected, and so on. If no matching catch is found, then the default exception handler halts the program and prints the stack trace.

throws

A throws clause lists the types of exceptions that a method might throw. This is necessary for all exceptions; The Java Language type Error or RuntimeException, or any of their subclasses. All other exceptions that a method can throw must be declared in the throws clause. If they are not, a compile-time error will result. This is the general form of a method declaration that includes a throws clause:

```
type method-name(parameter-list) throws exception-list { // body of method }
```

Here, exception-list is a comma-separated list of the exceptions that a method can throw.

16 a) Write a Java program that reads a binaryfile and write to another file.

```
import java.io.*;

public class FileCopyBinary
{
    public static void main(String[] args)
    {
        try {
            InputStream inputStream = new FileInputStream("inputfile.txt");
            OutputStream outputStream = new FileOutputStream("outputfile.txt");
            byte[] buffer = new byte[4096];
            int bytesRead = -1;
            while ((bytesRead = inputStream.read(buffer)) != -1) {
```

```

outputStream.write(buffer, 0, bytesRead);
}
} catch (IOException ex) {
ex.printStackTrace();
}
}
}
}

```

b) Write Java code that reads a character file and prints the contents of file on the display, with a line number before each line.

```

import java.io.*;
import java.util. Scanner;
public class FilePgm
{
public static void main(String args[])
{
int i=1;
try
{
//the file to be opened for reading
FileInputStream fis=new FileInputStream("inputfile.txt");
Scanner sc=new Scanner(fis);//file to be scanned
//returns true if there is another line to read
while(sc.hasNextLine())
{
System.out.print(i+" ");
System.out.println(sc.nextLine());//returns the line that was skipped
i++;
}
sc.close();//closes the scanner
}
}

```



```

catch(IOException e)
{
e.printStackTrace();
}
}
}

```

17 a) Write a Java program to find the duplicate characters in a string.

```

public class CountDuplicate
{
public static void main(String[] args)
{
String string1 = "Great responsibility";
int count;
//Converts given string into character array
char string[] = string1.toCharArray();
System.out.println("Duplicate characters in a given string: ");
//Counts each character present in the string
for(int i = 0; i <string.length; i++)
{
count = 1;
for(int j = i+1; j <string.length; j++)
{
if(string[i] ==string[j] && string[i]!=' ')
{
count++;
//Set string[j] to O to avoid printing visited character
string[j] = 'O';
}
}
}
}
}

```

```
//A character is considered as duplicate if count is greater than 1
if(count > 1 && string[i]!='O')
System.out.println(string[i]);
}
}
}
```

Output

Duplicate characters in a given string:

```
r
e
t
s
i
```

b) What are the uses of synchronized keyword in Java? Explain with examples.

- When we start two or more threads within a program, there may be a situation when multiple threads try to access the same resource and finally they can produce unforeseen result due to concurrency issues.
- Synchronization in java is the capability of controlling the access of multiple threads to any shared resource.
- For example, if multiple threads try to write within a same file then they may corrupt the data because one of the threads can override data or while one thread is opening the same file at the same time another thread might be closing the same file.
- So there is a need to synchronize the action of multiple threads and make sure that only one thread can access the resource at a given point in time.

a)using synchronized method

- With a synchronized method, the lock is obtained for the duration of the entire method.
- So if you want to lock the whole object, use a synchronized method

synchronized void methodname ()

eg: implementation of synchronized method

```
class First{
synchronized public void display(String msg) {
```

```

System.out.print("[ "+msg);
try {
Thread.sleep(500);
}
catch(InterruptedException e) {
e.printStackTrace();
}
System.out.println("]");
}}

class Second extends Thread {
String msg;
First fobj;
Second (First fp,String str) {
fobj = fp;
msg= str;
start();
}
public void run() {
fobj.display(msg);
} }

public class MyThreadDemo
{
public static void main (String[] args)
{
First fnew = new First();
Second ss = new Second(fnew,"welcome");
Second ss1= new Second(fnew,"new");
Second ss2= new Second(fnew, "programmer");
} }

```

b)Using Synchronized block

- If we want to synchronize access to an object of a class or only a part of a method to be synchronized then we can use synchronized block for it.
- It is capable to make any part of the object and method synchronized.
- With synchronized blocks we can specify exactly when the lock is needed. If you want to keep other parts of the object accessible to other threads, use synchronized block.
- General form of the synchronized statement :

synchronized(object identifier)

```
{ // Access shared variables and other shared resources  
}
```

Eg:

```
class First{  
    public void display(String msg) {  
        System.out.print "["+msg);  
        try {  
            Thread.sleep(1000);  
        }  
        catch (InterruptedException e) {  
            e.printStackTrace();  
        }  
        System.out.println("]");  
    }  
}  
  
class Second extends Thread {  
    String msg;  
    First fobj;  
    Second (First fp,String str) {  
        fobj = fp;  
        msg= str;  
        start();  
    }  
    public void run() {
```

```

synchronized(fobj)//synchronized block
{
    fobj.display(msg);
} } }

public class SyncBlock
{
    public static void main (String[] args)
    {
        First fnew = new First();
        Second ss = new Second(fnew,"welcome");
        Second ss1= new Second(fnew,"new");
        Second ss2= new Second(fnew, "programmer");
    } }

```

18 a) Write a Java program that creates three threads. First thread generates a random positive number (>1) every 1 second. If the number is even, the second thread prints all even numbers between 1 and the generated number. If the number is odd, the third thread will print all odd numbers between 1 and the generated number.

```

import java.util.Random;

class RandomNumberThread extends Thread {

    public void run() {

        Random random = new Random();

        for (int i = 0; i < 10; i++) {

            int randomInteger =random.nextInt(100);

            System.out.println("Random Integer generated : " + randomInteger );

            if((randomInteger %2)==0)

            {

                EvenThread sThread = new EvenThread(randomInteger );

                sThread.start();

            }

            else {

```

```
OddThread cThread = new OddThread(randomInteger);  
cThread.start();  
}
```

```
try {  
    Thread.sleep(2000);  
}  
catch (InterruptedException ex) {  
    System.out.println(ex);  
}  
}  
}  
}
```

```
class EvenThread extends Thread {  
    int number;  
    EvenThread(int randomNumber) {  
        number = randomNumber;  
    }  
    public void run() {  
        for(int i=2;i<number;i=i+2)  
            System.out.println(i+" ");  
    }  
}
```

```
class OddThread extends Thread {  
    int number;  
    OddThread (int randomNumber) {  
        number = randomNumber;  
    }  
    public void run() {
```

```
for(int i=1;i<number;i=i+2)
System.out.println(i+" ");
}
}
```

```
public class ThreadRandom {
public static void main(String args[]) {
RandomNumberThread rnThread = new RandomNumberThread();
rnThread.start();
}
}
```

Output

Random Integer generated : 36

2
4
6
8
10
12
14
16
18
20
22
24
26
28
30
32

34

Random Integer generated : 7

1

3

5

Random Integer generated : 10

2

4

6

8

b) Differentiate between Collection Interface and Collections Class.

Collection: Collection is a interface present in java.util.package. It is used to represent a group of individual objects as a single unit. It is similar to the container in the language. The collection is considered as the root interface of the collection framework. It provides several classes and interfaces to represent a group of individual objects as a single unit.

The List, Set, and Queue are the main sub-interfaces of the collection interface. The map interface is also part of the java collection framework, but it doesn't inherit the collection of the interface. The add(), remove(), clear(), size(), and contains() are the important methods of the Collection interface.

Declaration:

```
public interface Collection<E> extends Iterable<E>
```

Type Parameters: E - the type of elements returned by this iterator

Collections: Collections is a utility class present in java.util.package. It defines several utility methods like sorting and searching which is used to operate on collection. It has all static methods. These methods provide much-needed convenience to developers, allowing them to effectively work with Collection Framework. For example, It has a method sort() to sort the collection elements according to default sorting order, and it has a method min(), and max() to find the minimum and maximum value respectively in the collection elements.

Declaration:

```
public class Collections extends Object
```


Collection	Collections
It is an interface.	It is a utility class.
It is used to represent a group of individual objects as a single unit.	It defines several utility methods that are used to operate on collection.
The Collection is an interface that contains a static method since java8. The Interface can also contain abstract and default methods.	It contains only static methods.

19 a) How do you establish connection between a Java program and database?

Explain the steps with sample code.

Basic Steps to use a Database in Java

- 1.Establish a connection
- 2.Create JDBC Statements
- 3.Execute SQL Statements
- 4.Get ResultSet
- 5.Close connections

1. Establish a Connection

```
import java.sql.*;
```

Load the vendor specific driver

```
o Class.forName("oracle.jdbc.driver.OracleDriver");
```

Dynamically loads a driver class, for Oracle database

Make the connection

```
o Connection con =
```

```
DriverManager.getConnection("jdbc:oracle:thin:@oracle-prod: 1521 :OPROD", username, passwd);
```

Establishes connection to database by obtaining a Connection object

2. Create JDBC Statements

```
Statement stmt = con.createStatement();
```

Creates a Statement object for sending SQL statements to the database

3. Execute SQL Statements

```
String createLehigh = "Create table Lehigh " + "(SSN Integer not  
null, Name VARCHAR(32), " + "Marks Integer)";  
stmt.executeUpdate(createLehigh);  
String insertLehigh = "Insert into Lehigh values" 100);"  
stmt.executeUpdate(insertLehigh);
```

4. Get ResultSet

```
String queryLehigh = "select * from Lehigh";  
ResultSet rs = Stmt.executeQuery(queryLehigh);  
//What does this statement do?  
while (rs.next()) {  
    int ssn = rs.getInt("SSN");  
    String name = rs.getString("NAME");  
    int marks = rs.getInt("MARKS");
```

5. Close Connection

```
stmt.close();  
con.close();  
rs.close()
```

b) Write java code to demonstrate the execution of select and delete queries using JDBC

```
// Java program to illustrate selecting from Database  
import java.sql.*;  
public class select  
{  
    public static void main(String args[])  
    {  
        String id = "id1 ";
```

```

String pwd = "pwd1 ";

try
Class.forName("oracle.jdbc.driver.OracleDriver");

Connection con = DriverManager.getConnection("jdbc:oracle:thin:@localhost:1521:orcl", "login1",
"pwd1");

Statement stmt = con.createStatement();

// SELECT query

String q1 = "select * from userid WHERE Id = '"+id+"' AND pwd = '"+pwd+"' ";

ResultSet rs = stmt.executeQuery(q1);

if (rs.next())
{
System.out.println("User-Id : " + rs.getString(1));
System.out.println("Full Name : " + rs.getString(3));
System.out.println("E-mail : " + rs.getString(4));
}
else
{
System.out.println("No such user id is already registered");
}

//Deleting from database

string q1 = "DELETE from userid WHERE Id='"+id+"'AND pwd = '"+pwd+"'";

int x = stmt.executeUpdate(q1 );

if(x > 0)

System.out.println("One User Successfully Deleted");

else

System.out.println("Error Occurred");

con.close();

}

catch(Exception e)

{

System.out.println(e);

```

```
}  
}  
}
```

20 a) How events are handled in java Swing?

Event Handling is the mechanism that controls the event and decides what should happen if an event occurs. This mechanism has a code which is known as an event handler, that is executed when an event occurs.

Java uses the Delegation Event Model to handle the events. This model defines the standard mechanism to generate and handle the events.

The Delegation Event Model has the following key participants.

Source — The source is an object on which the event occurs. Source is responsible for providing information of the occurred event to it's handler. Java provide us with classes for the source object.

Listener — It is also known as event handler. The listener is responsible for generating a response to an event. From the point of view of Java Implementation, the listener is also an object. The listener waits till it receives an event. Once the event is received, the listener processes the event and then returns.

The benefit of this approach is that the user interface logic is completely separated from the logic that generates the event. The user interface element is able to delegate the processing of an event to a separate piece of code.

In this model, the listener needs to be registered with the source object so that the listener can receive the event notification. This is an efficient way of handling the event because the event notifications are sent only to those listeners who want to receive them.

Steps Involved in Event Handling

Step 1 — The user clicks the button and the event is generated.

Step 2 — The object of concerned event class is created automatically and information about the source and the event get populated within the same object.

Step 3 — Event object is forwarded to the method of the registered listener class.

Step 4 — The method is gets executed and returns.

b) Write a Java program using Swing to create a frame having three textfields, three labels and a button. The interface has to accept a number in the first text field. While clicking the button, the second and third textfields have to display the previous number and next number respectively, of the accepted input number.

```

import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

class NumberFrame extends JFrame implements ActionListener
{
    JLabel textLabel, Label1, Label2;
    JTextField Text1, Text2, Text3;
    JButton ok, exit;
    JPanel pan1, pan2;
    int occurrences = 0, i = 0;

    public NumberFrame()
    {
        textLabel = new JLabel("Enter Number ",JLabel.CENTER);
        Label1 = new JLabel("Previous Number ",JLabel.CENTER);
        Label2 = new JLabel("Next Number ",JLabel.CENTER);
        Text1 = new JTextField(20);
        Text2 = new JTextField(20);
        Text3 = new JTextField(20);
        pan1 = new JPanel();
        pan1.setLayout(new GridLayout(4,2));
        pan1.add(textLabel);
        pan1.add(Text1);
        pan1.add(Label1);
        pan1.add(Text2);
        pan1.add(Label2);
        pan1.add(Text3);
        ok = new JButton("ok");
        exit = new JButton("Exit");
        ok.addActionListener(this);
        exit.addActionListener(this);
    }
}

```

```

    pan2 = new JPanel();
    pan2.setLayout(new FlowLayout());
    pan2.add(ok);
    pan2.add(exit);
    add(pan1,"Center");
    add(pan2,"South");
    setTitle("Numnber Operation");
    setSize(300, 200);
    setVisible(true);
}

public void actionPerformed(ActionEvent ae)
{
    JButton btn = (JButton)ae.getSource();
    if (btn == ok)
    {
        String f = Text1.getText();
        int s=Integer.parseInt(f);
        s=s+1;
        Text3.setText(Integer.toString(s));
        int a=Integer.parseInt(f);
        a=a-1;
        Text2.setText(Integer.toString(a));
    }
    if (btn == exit)
    {
        System.exit(0);
    }
}

public static void main(String[] args)
{

```

```
        new NumberFrame();  
    }  
}
```